University course pack

Teaching cyber security at university

A 14-week, 3-credit module in computer and network security for third-year computer science or computer engineering students. Prerequisites: programming, discrete mathematics and networking. The four Darsi lectures anchor weeks 2, 5, 7 and 9.

Learning outcomes

1. Apply classic secure-design principles to evaluate a system.
2. Produce a STRIDE threat model with prioritised mitigations.
3. Explain what key exchange, encryption and signatures each guarantee.
4. Configure password storage and phishing-resistant authentication.
5. Conduct offensive exercises legally and ethically within a signed scope.

14-week syllabus

Week	Topic	In class and out
1	Security goals, law and ethics	Sign the rules of engagement
2	Design principles and threat modelling	Lab 1: STRIDE on the course portal
3	Symmetric cryptography	Problem set 1
4	Number theory for cryptography	Paper exercises
5	Diffie–Hellman and RSA	Lab 2: man-in-the-middle
6	PKI, certificates and TLS	Problem set 2
7	Hashing and password storage	Lab 3: cracking synthetic tables
8	Authentication and MFA	Midterm exam
9	Offensive labs and responsible disclosure	Lab 4: isolated range
10	Web application security	CTF challenges 1 and 2
11	Network attacks and defences	CTF challenges 3 and 4
12	Social engineering and phishing	CTF challenge 5 and write-ups
13	Incident response	Tabletop exercise
14	Review and project presentations	Final exam

University course pack

Lecture slide outline

Lecture 1: Principles and STRIDE

- A breach caused by a default
- Saltzer and Schroeder's eight principles
- Data-flow diagrams and trust boundaries
- STRIDE threats and properties
- Activity: model the portal

Lecture 2: Public-key cryptography

- The key-distribution problem
- Diffie–Hellman with small numbers
- RSA encryption and signatures
- Why padding matters
- Activity: Alice, Bob and Eve

Lecture 3: Passwords and authentication

- Hash properties and the avalanche effect
- Why fast hashes fail
- Salts and Argon2id
- SMS, apps and FIDO2 keys
- Activity: crack a fake table

Lecture 4: Ethics, labs and CTF

- Authorised testing versus intrusion
- Rules of engagement
- Building an isolated range
- Per-student flags and write-ups
- Activity: design a fair CTF

University course pack

Problem set

1. A file share is readable by all staff because it was easier to set up. Name two principles violated and propose a fix.

2. For a login flow (browser → web server → database), list one threat for each STRIDE letter.

3. With p = 23 and g = 5, Alice picks a = 6 and Bob picks b = 15. Compute A, B and the shared key.

4. With p = 5, q = 11 and e = 3, find d and encrypt m = 2.

5. An attacker tests 10^{10} SHA-256 guesses per second. How long to try a 10^8-word list against 10^4 unsalted accounts, and against 10^4 salted accounts?

6. A student reports finding an exposed admin page on the university website. Outline the course's expected response.

Solutions

1. Fail-safe defaults and least privilege. Deny by default and grant read access per group that needs it.

2. S: stolen session cookie. T: altered request parameters. R: no audit log of logins. I: SQL injection leaking hashes. D: login flooding. E: missing authorisation on admin routes.

3. $A = 5^6 \bmod 23 = 8$, $B = 5^{15} \bmod 23 = 19$, key $= 19^6 \bmod 23 = 8^{15} \bmod 23 = 2$.

4. $n = 55$, $\varphi(n) = 40$, $d = 27$ because $3 \times 27 = 81 \equiv 1 \bmod 40$. $c = 2^3 \bmod 55 = 8$.

5. Unsalted: hash the list once, $10^8 / 10^{10} = 0.01$ s. Salted: once per account, $10^4 \times 0.01 = 100$ s. Salts multiply work by the number of accounts; slow hashes multiply it further.

6. Stop testing, do not access further, report privately to the lecturer and IT security with details, keep it confidential until fixed, and recognise the student for responsible disclosure.

University course pack

Lab handout: Lab 3: Password storage under attack

Goal

Measure how salts and slow hashes change an attacker's cost, using only synthetic accounts in an isolated VM.

Steps

1. Generate 1,000 synthetic accounts from a public word list.
2. Store them with SHA-256, salted SHA-256 and Argon2id.
3. Run the same dictionary attack on each table and record guesses per second.
4. Tune Argon2id so one verification takes about 100 ms.
5. Write a storage and MFA policy for a university system.

Hand in

A results table, the tuned parameters and a one-page policy.

University course pack

Assessment rubric

Criterion	Excellent	Good	Developing	Not yet
Threat analysis	Covered and prioritised by risk	Every element covered	STRIDE applied partly	Few threats, no structure
Cryptographic reasoning	Spots protocol flaws	States guarantees	Computes correctly	Confuses operations
Defensive fix	Specific, tested and justified	Specific and correct	Generic	None
Professional conduct	Documented, with responsible disclosure	Within scope, documented	Within scope	Outside scope

Reading list

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278–1308.
The design principles.

Shostack, A. (2014). Threat Modeling: Designing for Security. Wiley.
STRIDE in practice.

Diffie, W., & Hellman, M. E. (1976). New directions in cryptography. IEEE Transactions on Information Theory, 22(6), 644–654.
Public-key key exchange.

Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120–126.
RSA.

Katz, J., & Lindell, Y. (2020). Introduction to Modern Cryptography (3rd ed.). CRC Press.
Rigorous core text.

Anderson, R. (2020). Security Engineering (3rd ed.). Wiley.
Case studies of real failures.

Temoshok, D., et al. (2025). NIST Special Publication 800-63B-4: Digital Identity Guidelines, Authentication and Authenticator Management. NIST.
Password and authenticator guidance.

Joint Task Force on Cybersecurity Education (2017). Cybersecurity Curricula 2017 (CSEC2017). ACM/IEEE-CS/AIS SIGSEC/IFIP WG 11.8.
Curriculum guidance.